Introduction

Low-Level Design (LLD) interviews evaluate your ability to transform real-world scenarios into clean, scalable, and easy-to-maintain code structures. One of the most commonly discussed examples is the **Hotel Management System**.

In this article, we'll guide you through a clear step-by-step framework to design this system, just like you would in a real interview. Along the way, we'll share pro tips to help you articulate your reasoning, handle edge cases, and impress your interviewer with a well-thought-out approach.

Step 1: Clarify Requirements

Functional Requirements:

- The system should allow users to search and browse hotels/rooms by location (city/geo), date range (check-in, check-out), room type (capacity/bed type/amenities), and price range
- The system should provide real-time availability check per night, per hotel, per room type
- The system should support booking cancellations with policies (non-refundable, partial refund with cutoff)
- The system should handle payment workflow with states (Pending → Completed → Refunded/Failed)
- The system should support dynamic daily/seasonal pricing where price can vary per day
- The system should support multiple roles: Customer, Admin
- The system should provide user dashboard to view past and upcoming bookings
- The system should allow users to reserve a room (single room per booking)

- The system should provide Admin panel to add/remove/update hotel/room/pricing/policies
- The system should have a background scheduler to process expired HELD bookings and restore inventory

Interview Tip: *Always ask the interviewer to confirm the requirements before jumping into code. It shows clarity and alignment.*

Edge Cases to Consider

- Cancellation policy on hotel or room?
- Overbooking allowed? If yes, how much?
- How to show price for multiple days if it includes a surge day?
- Do we maintain check in and check out for the booking made?
- What if the user checks out before check out date?
- Any coupon code management?

Step 2: Identify Core Entities

1. Hotel (Core Entity)

- id: String [PK]
- name: String
- address: String
- city: String
- country: String
- latitude: double
- longitude: double
- rating: double

- isActive: boolean
- defaultOverbookPercent: int (e.g., 10)
- cancellationPolicyId: String [FK]
- createdAt: long

2. RoomType

- id: String [PK]
- hotelId: String [FK]
- name: String (e.g., Deluxe King)
- capacity: int
- bedType: String (e.g., KING, QUEEN, TWIN)
- basePrice: long (in minor units)
- amenities: List<String>
- totalRooms: int
- isActive: boolean
- createdAt: long

3. Room

- id: String [PK]
- hotelId: String [FK]
- roomTypeId: String [FK]
- roomNumber: String
- isActive: boolean
- createdAt: long

4. Booking

- id: String [PK]
- userId: String [FK]
- hotelId: String [FK]
- roomTypeId: String [FK]
- checkInDateUtc: long
- checkOutDateUtc: long
- nightlyPrices: List<NightlyPrice>

- totalAmountMinor: long
- bookingStatus: BookingStatus (CREATED, HELD, CONFIRMED, CHECKED_IN, CHECKED_OUT, CANCELLED)
- paymentStatus: TransactionStatus (PENDING, COMPLETED, REFUNDED, FAILED)
- allocatedRoomId: String (nullable)
- checkInTimeUtc: long (nullable)
- checkOutTimeUtc: long (nullable)
- holdExpiresAt: long
- createdAt: long

5. Transaction

- id: String [PK]
- bookingId: String [FK]
- amountMinor: long
- currency: String
- status: TransactionStatus (PENDING, COMPLETED, REFUNDED, FAILED)
- providerRef: String
- createdAt: long
- completedAt: long (nullable)
- refundedAt: long (nullable)

6. SeasonalPrice (Dynamic Daily Price)

- id: String [PK]
- hotelId: String [FK]
- roomTypeId: String [FK]
- dateUtc: long
- priceMinor: long
- createdAt: long

7. CancellationPolicy

- id: String [PK]
- name: String (NON_REFUNDABLE, PARTIAL, FLEX)

- refundPercent: int (0..100)
- cutoffHoursBeforeCheckIn: int
- createdAt: long

8. User

- id: String [PK]
- name: String
- email: String
- role: UserRole (CUSTOMER, ADMIN)
- createdAt: long

9. SearchFilter (Request DTO)

10. RoomTypeAvailability (Response DTO)

- roomTypeId: String
- roomTypeName: String
- capacity: int
- bedType: String
- amenities: List<String>
- available: boolean
- totalPrice: long
- averagePricePerNight: double
- nightlyPrices: List<NightlyPrice>

ENUMS

- BookingStatus: CREATED, HELD, CONFIRMED, CHECKED_IN, CHECKED_OUT, CANCELLED
- TransactionStatus: PENDING, COMPLETED, REFUNDED, FAILED
- UserRole: CUSTOMER, ADMIN

Interview Tip: *Clearly defining entities ensures smooth implementation of pricing, availability, booking workflow, payments, and cancellations in the hotel reservation system.*

Step 3: Visualize Interaction Flows

1. Hotel Search & Browse:

- User enters filters → System queries search index → Returns hotels/room types with total price and average price per night

2. Real-time Availability:

- User selects hotel + date range → System queries all room types for hotel
- For each room type: checks availability (query CONFIRMED and HELD bookings, CREATED bookings don't count)
- Calculates pricing → Returns List<RoomTypeAvailability> with only available room types + details and pricing

3. Booking (Two-Phase):

Phase 1 - Create Booking:

- Validate request → Precheck availability (CONFIRMED + HELD)
- Fetch current prices (SeasonalPrice, fallback basePrice)
- Validate expected price → Price the stay
- Create booking (CREATED, paymentStatus=PENDING) with locked prices
- No inventory reduction (CREATED doesn’t affect availability)

Phase 2 - Initiate Payment:

- Initiate payment → Validate booking is CREATED
- Update (CREATED → HELD) → Reduce inventory
- On payment success → Booking HELD → CONFIRMED

- On payment failure/timeout → Cancel booking → Restore inventory

4. Cancellation:

- User requests cancel → Validate booking cancellable
- Apply policy by time window → Mark CANCELLED
- Trigger refund if applicable
- Inventory auto-restored (CANCELLED doesn't count)

5. Check-in:

- Admin checks in guest → Assign room
- Update booking to CHECKED_IN → Set checkInTimeUtc + allocatedRoomId

6. Check-out:

- Admin checks out guest → Update to CHECKED_OUT
- Set checkOutTimeUtc
- If early check-out → Release inventory for remaining dates

7. Admin:

- CRUD hotel/room/roomType
- Adjust inventory/overbooking percent
- Manage seasonal prices and cancellation policies

8. User Dashboard:

- View past vs upcoming bookings with details and receipts

9. Background Scheduler:

- Runs every 1–5 minutes
- Finds expired HELD bookings → Marks CANCELLED → Restores inventory

---

Step 4: Defines Class Structures & Relationships

Layers of Architecture

We will design the **architecture layers** of the system in a structured way, ensuring **separation of concerns** and **modularity**. The system will be organized into the following layers:

Client/UI Layer → Controller Layer → Service Layer → Domain Layer

---

CONTROLLERS:

1. SearchController (Search & Availability)

- - List<Hotel> searchHotels(SearchFilter filter)
- - List<RoomTypeAvailability> getAvailability(String hotelId, DateRange range)

2. BookingController (Reservation)

- - Booking createBooking(String userId, String hotelId, String roomTypeId, DateRange range, long expectedTotalPrice)
- - void cancelBooking(String bookingId, String userId)

3. TransactionController (Transactions)

- - Transaction initiateTransaction(String bookingId)
- - void handleTransactionCallback(String providerRef, TransactionStatus status)

4. AdminController (Admin)

- - Hotel createOrUpdateHotel(Hotel hotel)
- - RoomType createOrUpdateRoomType(RoomType roomType)

- - void updateOverbookingPercent(String hotelId, String roomTypeId, int percent)
- - SeasonalPrice setSeasonalPrice(String hotelId, String roomTypeId, long dateUtc, long priceMinor)
- - CancellationPolicy createOrUpdatePolicy(CancellationPolicy policy)
- - Booking checkIn(String bookingId, String roomId, long checkInTimeUtc)
- - Booking checkOut(String bookingId, long checkOutTimeUtc)

5. DashboardController (User)

- - List<Booking> listUserBookings(String userId)

SERVICES:

1. SearchService

- - List<Hotel> searchHotels(SearchFilter filter)
- - List<RoomTypeAvailability> getAvailability(String hotelId, DateRange range)

2. InventoryService

- - int getConfirmedBookingsCount(String hotelId, String roomTypeId, long dateUtc)
- - int getHeldBookingsCount(String hotelId, String roomTypeId, long dateUtc)
- - int getCheckedInBookingsCount(String hotelId, String roomTypeId, long dateUtc)
- - boolean checkAvailability(String hotelId, String roomTypeId, DateRange range, int qty)

3. PricingService

- - List<NightlyPrice> rateStay(String hotelId, String roomTypeId, DateRange range)
- - long computeTotal(List<NightlyPrice> nightly)
- - double computeAveragePricePerNight(List<NightlyPrice> nightly, int numberOfNights)

4. BookingService

- - Booking createBooking(String userId, String hotelId, String roomTypeId, DateRange range, long expectedTotalPrice)
- - void cancelBooking(String bookingId, String userId)
- - Booking checkIn(String bookingId, String roomId, long checkInTimeUtc)
- - Booking checkOut(String bookingId, long checkOutTimeUtc)

5. TransactionService

- - Transaction initiateTransaction(String bookingId)
- - void handleCallback(String providerRef, TransactionStatus status)
- - void issueRefund(String bookingId, long amountMinor)

6. BookingStateHandler (State Pattern Implementation)

- - boolean canTransition(BookingStatus current, BookingStatus newStatus)
- - void transition(Booking booking, BookingStatus newStatus)
- - void requireStatus(Booking booking, BookingStatus expectedStatus)
- - void requireAnyStatus(Booking booking, BookingStatus... allowedStatuses)
- - boolean canCancel(Booking booking)
- - boolean canCheckIn(Booking booking)
- - boolean canCheckOut(Booking booking)
- - boolean canInitiateTransaction(Booking booking)
- - boolean countsInInventory(Booking booking)

7. PolicyService

- - RefundDecision evaluateCancellation(Booking booking, CancellationPolicy policy, long nowUtc)

8. UserService

- - List<Booking> listUserBookings(String userId)

9. SchedulerService (Background Jobs)

- - void processExpiredHolds()
- - List<Booking> findExpiredHeldBookings(long nowUtc)

REPOSITORIES:

1. HotelRepository

- - Hotel save(Hotel hotel)
- - Optional<Hotel> findById(String hotelId)
- - List<Hotel> findByLocation(String city, String country)

2. RoomTypeRepository

- - RoomType save(RoomType roomType)
- - Optional<RoomType> findById(String roomTypeId)
- - List<RoomType> findByHotel(String hotelId)

3. RoomRepository

- - Room save(Room room)
- - List<Room> findByHotelAndType(String hotelId, String roomTypeId)

4. BookingRepository

- - Booking save(Booking booking)
- - Optional<Booking> findById(String bookingId)
- - List<Booking> findByUser(String userId)

- - int countConfirmedBookings(String hotelId, String roomTypeId, long dateUtc)
- - int countHeldBookings(String hotelId, String roomTypeId, long dateUtc, long nowUtc)
- - int countCheckedInBookings(String hotelId, String roomTypeId, long dateUtc)

5. TransactionRepository

- - Transaction save(Transaction transaction)
- - Optional<Transaction> findByProviderRef(String providerRef)

6. SeasonalPriceRepository

- - SeasonalPrice upsert(SeasonalPrice price)
- - Optional<SeasonalPrice> findByKey(String hotelId, String roomTypeId, long dateUtc)

7. CancellationPolicyRepository

- - CancellationPolicy save(CancellationPolicy policy)
- - Optional<CancellationPolicy> findById(String policyId)

PATTERNS & KEY RELATIONSHIPS:

- **State** – BookingStatus and TransactionStatus drive transitions and side effects
- **Repository** – Data access abstraction
- **Service Layer** – Business logic separation
- **Database Transactions** – Use transactions with row-level locking for conflict-free booking
- **Domain Events** – Transaction completion → booking confirmation; cancellation → refund/inventory release

---

Step 5: Core Use Cases & Methods

1. Hotel Search:

- User enters filters → SearchService.searchHotels(filter) →
- Fetch hotels by location
- Optionally prefetch top room types
- Calculate total price & average price per night via PricingService.rateStay(), computeTotal(), computeAveragePricePerNight()
- Return hotels with total price and avg price per night

---

2. Real-time Availability (Returns All Available Room Types):

- getAvailability(hotelId, range) →
- Get all room types: RoomTypeRepository.findByHotel(hotelId)
- For each room type:
    - Get totalRooms + overbookPercent
    - For each date in range:
        - Query countConfirmedBookings()
        - Query countHeldBookings()
        - Query countCheckedInBookings()
    - Compute available = totalRooms + overbook - confirmed - held - checkedIn
    - If available ≥ 1 → compute pricing using PricingService
    - Build RoomTypeAvailability
- Return List<RoomTypeAvailability>

---

3. Booking Flow (Two-Phase - Single Room):

Phase 1: Create Booking

- createBooking(userId, hotelId, roomTypeId, range, expectedTotalPrice)
- Validate inputs (user, roomType, date range)
- InventoryService.checkAvailability()
- PricingService.rateStay() to fetch nightly price (SeasonalPrice → fallback to basePrice)
- Calculate total → validate against expectedTotalPrice
- Create booking with CREATED status and locked nightly prices
- Return booking (CREATED does not reduce inventory)

Phase 2: Initiate Payment

- initiateTransaction(bookingId)
- Validate booking status = CREATED
- Re-check availability
- Create transaction with status=PENDING
- Set booking status → HELD, set holdExpiresAt=now+TTL
- Inventory reduces (HELD counts in availability)
- Return transaction

Payment Callback

- On SUCCESS → booking HELD → CONFIRMED, paymentStatus=COMPLETED
- On FAILURE/TIMEOUT → booking HELD → CANCELLED, inventory restored

Hold Expiry (Background Scheduler)

- Find HELD bookings where holdExpiresAt < now

- Set status=CANCELLED, paymentStatus=FAILED
- Inventory restored

---

4. Cancellation & Refund:

- cancelBooking(bookingId, userId)
- Validate ownership + disallow if CHECKED_IN/CHECKED_OUT
- Get cancellation policy from hotel
- PolicyService.evaluateCancellation() → RefundDecision
- Update BookingStatus=CANCELLED; if refund>0 → paymentStatus=REFUNDED
- Issue refund via TransactionService.issueRefund()
- Inventory restored for all dates in stay

---

5. Check-in Use Case:

- checkIn(bookingId, roomId, checkInTimeUtc)
- Validate booking is CONFIRMED
- Assign room
- Update status=CHECKED_IN; set checkInTimeUtc

---

6. Check-out Use Case:

- checkOut(bookingId, checkOutTimeUtc)
- Validate booking is CHECKED_IN
- Set status=CHECKED_OUT, store checkOutTimeUtc
- If early checkout → release inventory for remaining nights

---

7. Admin Management:

- Create/update Hotel, RoomType, Room
- Set default or room-type-level overbookingPercent
- Manage SeasonalPrice (per day)
- Create/update CancellationPolicy and assign to Hotel

---

8. User Dashboard:

- listUserBookings(userId)
- Separate past vs upcoming using check-in/check-out timestamps

---

Step 6: Apply OOP Principles & Design Patterns

Design Patterns Used:

- **Repository Pattern –** for data access abstraction
- **Service Layer Pattern –** for business logic separation
- **State Pattern –** BookingStateHandler manages all BookingStatus transitions and validations
    - Encapsulates state transition logic (no manual if/else checks)
    - Validates transitions: CREATED→HELD, HELD→CONFIRMED, HELD→CANCELLED, CONFIRMED→CHECKED_IN, CHECKED_IN→CHECKED_OUT, etc.
    - Provides helper methods: canCancel(), canCheckIn(), canCheckOut(), canInitiateTransaction(), countsInInventory()

    - All services use BookingStateHandler.transition() and BookingStateHandler.requireStatus() instead of manual checks
- **Controller Separation –** clear responsibilities by domain
- **Domain Events –** decouple payment, booking, inventory, notifications

---

OOP Principles Applied:

- **Single Responsibility –** each class handles one concern
- **Open/Closed –** easy to extend with new features without modifying existing code
- **Encapsulation –** domain objects control invariants (date range, totals)
- **Dependency Inversion –** services depend on repositories/interfaces
- **Consistency Boundaries –** inventory operations are the concurrency boundary

---

Step 7: Handle Edge Cases

Edge Case Solutions:

- **Double-booking under concurrency:**
    - Precheck availability before creating booking
    - Use database transactions when creating HELD booking to ensure atomicity
    - On payment initiation: verify booking still HELD and availability exists within transaction
    - HELD bookings count toward inventory, preventing double-booking

- **Payment initiation and inventory locking:**
    - Booking is already HELD when payment is initiated (counts in inventory)
    - On payment success: HELD → CONFIRMED
    - On payment failure/timeout: HELD → CANCELLED (inventory restored)
    - Hold expiry scheduler auto-cancels expired HELD bookings
- **Overbooking cap:**
    - overbookAllowed = ceil(totalRooms * overbookPercent / 100)
    - available = totalRooms + overbookAllowed - confirmedCount - heldCount - checkedInCount
    - Prevent booking when crossing the cap
- **Early check-out:**
    - CHECKED_OUT bookings stop counting toward availability
    - Remaining nights become available immediately
- **Partial availability across nights:**
    - All nights must be available or request fails
- **Pricing drift during checkout:**
    - Fetch current prices during booking creation
    - Validate fetched prices vs expected prices (if provided)
    - Lock nightly prices inside booking for TTL
    - Reject booking if any price mismatch
- **Payment callbacks:**
    - Callback handler checks transaction status before updating booking
    - Handles late or duplicate callbacks safely
    - Restores inventory if payment fails
- **Timezones & date math:**
    - Store dates in UTC
    - Night range uses [check-in, check-out) format
- **Cancellation and inventory restoration:**
    - Only HELD and CONFIRMED bookings can be cancelled
    - Status becomes CANCELLED

    - CANCELLED bookings don't count toward availability
    - All dates in stay range become available again
- **Cancellations near cutoff:**
    - Use nowUtc vs check-in minus cutoffHours
- **Admin edits while users checkout:**
    - Inventory & prices snapshot at hold time; future changes affect new holds only
- **Assigned room vs room type:**
    - Book by room type
    - Allocate actual room closer to check-in

Summary

This simplified design focuses on 6 core API flows perfect for a 60-minute interview:

- **Create Booking Hold:** POST /api/bookings/hold (creates HELD booking with price lock & inventory check)
- **Confirm Payment:** POST /api/bookings/{bookingId}/confirm-payment (payment success → CONFIRMED)
- **Cancel Booking:** POST /api/bookings/{bookingId}/cancel (restores inventory)
- **Check-in:** POST /api/bookings/{bookingId}/check-in (validates CHECKED_IN transition)
- **Check-out:** POST /api/bookings/{bookingId}/check-out (early or normal checkout → releases remaining nights)
- **Get Availability:** GET /api/hotels/{hotelId}/room-types/{roomTypeId}/availability?date=UTC (handles overbooking logic)

Key Features Implemented:

- Inventory-safe booking system with HELD, CONFIRMED, CHECKED_IN, CHECKED_OUT workflow
- Strict state machine using BookingStateHandler for all transitions
- Database transactions ensure atomic booking creation under concurrency
- Overbooking support with configurable hotel-level overbook percentage
- Price locking per night at hold-time to avoid pricing drift
- Automatic expiry of HELD bookings via scheduled job
- Early checkout support releasing remaining nights
- UTC-based date handling with [check-in, check-out) night ranges
- Consistent availability calculation across confirmed, held, and checked-in bookings
- Separation of concerns across controllers, services, repositories, and state handler

---

Design Pattern Highlights:

- **Repository Pattern –** for all data access (Hotels, RoomTypes, Bookings, Transactions, Prices)
- **Service Layer Pattern –** business logic fully separated from controllers
- **State Pattern –** BookingStateHandler manages all BookingStatus transitions and validations
- **Domain Events –** payment success/failure triggers booking transitions & inventory updates
- **Controller Separation –** clean division into BookingController, HotelController, TransactionController, etc.

- **Encapsulation –** domain models enforce invariants like date ranges & nightly pricing